

LLD-09: Python Advanced-1 — Typing and Generics

LLD-09: Python Advanced-1 — Typing and Generics

Python lets you write code without types. That's a feature for scripts. It's a liability for systems.

Recap — Concurrency Module

We spent 4 classes on concurrency. Before moving on, these concepts should be crystal clear.

Question 1: You have a shared counter accessed by 10 threads. The final value is wrong. Your teammate says “just use the GIL, Python is thread-safe.” What's wrong with their reasoning?

Answer

The GIL prevents **parallel execution** but NOT **context switching**. A thread can be interrupted BETWEEN reading and writing the counter. The GIL guarantees only that bytecode instructions are atomic — but `counter += 1` is NOT one bytecode instruction (it's LOAD, ADD, STORE). You still need a Lock/Mutex.

GIL ≠ thread safety. The GIL is about interpreter internals, not your application logic.

Question 2: A highway has 5 toll booths. 20 cars arrive. If you use a Mutex to control access, what happens? If you use `Semaphore(5)`, what happens?

Answer

Mutex: Only 1 car at a time. 4 booths sit empty. $20 \text{ cars} \times 1 \text{ second each} = 20 \text{ seconds}$.

Semaphore(5): 5 cars at a time. All booths used. $\text{ceil}(20/5) = 4 \text{ batches} \times 1 \text{ second} = 4 \text{ seconds}$.

Mutex is binary (0 or 1). Semaphore is a counter (0 to N). Use Semaphore when you have N identical resources.

Question 3: In Producer-Consumer, why can't a Mutex solve the "wait until buffer has space" problem? What can?

Answer

A mutex is binary — it's locked or unlocked. It can protect data but **cannot signal a condition**. If the producer holds the mutex and loops waiting for space, the consumer can never acquire the mutex to free space → **deadlock**.

A **Semaphore** can signal: producer does `space_sem.acquire()` (blocks when full), consumer does `space_sem.release()` (signals "space free"). The semaphore's `release()` WAKES UP a blocked `acquire()`. That's signalling — something a mutex fundamentally can't do.

Question 4: Your FastAPI app calls 3 external APIs and waits for all responses. Should you use threads or `async`? Why?

Answer

Async (`asyncio.gather` with `aiohttp`). FastAPI is already `async`. Using threads would add unnecessary overhead (thread creation, memory, GIL contention). With `async`, one thread handles all 3 requests concurrently — no context switching, no locks, minimal memory. Only use threads when your library is **blocking** (like requests or `psycopg2`).

Question 5: `time.sleep(1)` inside an `async def` function — what happens?

Answer

The entire event loop freezes for 1 second. No other coroutine can run. `time.sleep()` is a blocking call — it doesn't yield to the event loop. Use `await asyncio.sleep(1)` instead, which yields control and lets other tasks run during the wait. This is the #1 `async` bug.

Question 6: When would you choose `ProcessPoolExecutor` over `ThreadPoolExecutor`?

Answer

CPU-bound work — image processing, heavy math, data crunching. Threads don't help for CPU-bound work because the GIL prevents parallel execution on multiple cores. Processes bypass the GIL entirely — each process has its own Python interpreter and GIL. If your work is I/O-bound (network, disk), threads are sufficient and lighter.

Why Types? — The Problem

Python is **dynamically typed**. You can put anything anywhere:

```
def calculate_total(items):
    total = 0
    for item in items:
```

```
    total += item["price"] * item["quantity"]
    return total
```

Questions this code doesn't answer: - What is `items`? A list of dicts? A list of objects? What keys do the dicts have? - What types are `price` and `quantity`? `int`? `float`? `Decimal`? - What does the function return? `int`? `float`? Could it be `None`? - What happens if someone passes a string `discount` to `apply_discount(total, "10%")`?

You won't find out until production crashes at 3am.

The Bigger Picture

Codebase Size	Types?	Why
50-line script	Not needed	You can hold it all in your head
500-line project	Helpful	IDE autocomplete saves time
5000+ lines / team	Essential	Humans forget. Types don't.
Open-source library	Mandatory	Users can't read your mind

Companies that adopted typing: **Instagram** (3M lines of Python, typed with `mypy`), **Dropbox** (4M lines), **Stripe**, **Google**. All report 15-30% fewer production bugs after adding types.

Basic Type Hints

Function parameters and return type

```
def greet(name: str) -> str:
    return f"Hello, {name}!"
```

```
def add(a: int, b: int) -> int:
    return a + b
```

```
def is_adult(age: int) -> bool:
    return age >= 18
```

Python does NOT enforce types at runtime. Types are hints for: 1. **Your IDE** — autocomplete, warnings, navigation 2. **mypy** — static type checker, catches bugs before running 3. **Humans** — documentation that's always up-to-date

```
result = add("hello", " world") # Python runs this fine!
# But mypy says: error: Argument 1 has incompatible type "str";
expected "int"
```

Collection Types

Python 3.9+ syntax (preferred)

```
def process_names(names: list[str]) -> list[str]:
```

```

    return [name.upper() for name in names]

def get_scores() -> dict[str, int]:
    return {"Alice": 95, "Bob": 87}

def get_coordinates() -> tuple[float, float]: # fixed-length
    return (28.6139, 77.2090)

def get_all_ids() -> tuple[int, ...]: # variable-length
    return (1, 2, 3, 4, 5)

# Nested
def get_class_scores() -> dict[str, list[int]]:
    return {"Alice": [95, 88, 92], "Bob": [87, 91, 85]}

```

Note: Python 3.8 uses `from typing import List, Dict, Tuple`. Python 3.9+ lets you write `list[str]` directly. Always prefer the modern syntax.

Optional — “This value might not exist”

The #1 production crash: your code assumes a value exists, but it’s None.

```

def find_user(user_id: int) -> dict | None:
    users = {1: {"name": "Alice"}, 2: {"name": "Bob"}}
    return users.get(user_id) # returns None if not found

```

```

# Works in development:
user = find_user(1)
print(user["name"]) # "Alice" ✓

```

```

# 3am in production:
user = find_user(999)
print(user["name"]) # TypeError: 'NoneType' object is not subscriptable

```

`Optional[X]` is shorthand for `X | None`. Both mean: “this could be X, or it could be None.”

```

from typing import Optional

```

```

# These are IDENTICAL:
def find_user_v1(uid: int) -> Optional[dict]: ... # typing
    module syntax
def find_user_v2(uid: int) -> dict | None: ... # Python
    3.10+ pipe syntax

```

Three Ways to Handle Optional

```

# Pattern 1: if-check (most common)
user = find_user(999)
if user is not None:

```

```

    print(user["name"])    # mypy happy – inside the if, user is
                           dict
else:
    print("User not found")

# Pattern 2: early return (guard clause)
def get_user_name(user_id: int) -> str:
    user = find_user(user_id)
    if user is None:
        return "Unknown"    # bail out early
    return user["name"]    # mypy knows: user is dict here

# Pattern 3: raise if missing (Django/FastAPI pattern)
def get_user_or_404(user_id: int) -> dict:
    user = find_user(user_id)
    if user is None:
        raise ValueError(f"User {user_id} not found")
    return user    # mypy knows: user is dict

```

Common pitfall: `Optional[str] = None` means the parameter is optional to provide AND the value might be `None`. `str = "World"` means the parameter is optional but is **never** `None`. Don't use `Optional` when there's always a value.

Where Optional shows up

Everywhere: `dict.get()`, DB queries returning no row, API fields that may be missing, config values that may be unset. Tony Hoare (who invented null) called it his “billion dollar mistake” — `Optional` makes it visible.

Union — Multiple Types

```

# Accepts int or str – must handle both
def format_id(id_value: int | str) -> str:
    if isinstance(id_value, int):
        return f"ID-{id_value:05d}"    # mypy knows: int in this
        branch
    return f"ID-{id_value}"    # mypy knows: str in this
    branch

# Syntax evolution (all equivalent):
# Python 3.9: Union[int, str] / Optional[str] = Union[str,
#             None]
# Python 3.10: int | str          / str | None    ← prefer this

```

mypy's type narrowing: When you write `if isinstance(x, int)` or `if x is not None`, mypy narrows the type inside that branch. It tracks your control flow.

TypedDict and dataclass — Structured Data

```
from typing import TypedDict
from dataclasses import dataclass

# TypedDict: for dict-shaped data (API responses, JSON)
class UserDict(TypedDict):
    name: str
    age: int
    email: str

# dataclass: for your own data structures (preferred)
@dataclass
class User:
    name: str
    age: int
    email: str

    def is_adult(self) -> bool:
        return self.age >= 18

# Frozen dataclass = immutable
@dataclass(frozen=True)
class Config:
    host: str = "localhost"
    port: int = 8000
```

Use Case	Choose
JSON/API responses	TypedDict
Your own data structures	@dataclass
Validation needed (FastAPI)	Pydantic BaseModel
Complex behavior	Regular class

Callable — Typing Functions That Accept Functions

In Python, functions are objects. You can pass them around like integers or strings. **Callable** is how you type that.

The problem

```
def do_math(a, b, func):
    """What should func look like? 1 arg? 2 args? What does it
       return?"""
    return func(a, b)
```

A new developer sees func — they have no idea what to pass without reading the implementation.

The fix: Callable[[inputs], output]

```
from typing import Callable
```

```
def do_math(a: int, b: int, func: Callable[[int, int], int]) ->
    int:
    #                                     ~~~~~~
    #                                     func takes (int, int), returns
        int
    return func(a, b)
```

```
do_math(5, 3, add)      # ✓ add takes (int, int) returns int
do_math(5, 3, multiply) # ✓ multiply takes (int, int) returns int
do_math(5, 3, len)      # ✗ mypy error! len takes 1 arg, not 2
```

Reading the syntax

Callable Type	Means	Example Function
Callable[[int, int], int]	Takes 2 ints, returns int	def add(a: int, b: int) -> int
Callable[[str], None]	Takes 1 string, returns nothing	def log(msg: str) -> None
Callable[[], str]	Takes nothing, returns a string	def greet() -> str
Callable[[dict], list[str]]	Takes a dict, returns list of strings	def get_keys(d: dict) -> list[str]

Where you'll see Callable

```
# Callbacks / event handlers
```

```
def on_click(handler: Callable[[int, int], None]) -> None: ...
```

```
# Retry logic
```

```
def retry(func: Callable[[], str], times: int = 3) -> str: ...
```

```
# Strategy pattern
```

```
def sort_users(users: list, key: Callable[[User], int]) ->
    list: ...
```

```
# Decorators
```

```
def timer(func: Callable[..., Any]) -> Callable[..., Any]: ...
```

In one sentence: Callable is the type hint for a function — just like int describes “a number”, Callable[[int, int], int] describes “a function that takes two ints and returns an int.”

Generics — The Big Idea

The Problem

```
class Box:
    def __init__(self, item):
        self.item = item
    def get(self):
        return self.item

box = Box(42)
result = box.get()  # What type is result? IDE says: Any
result.upper()      # Runtime crash. IDE didn't warn.
```

Options: - **Separate classes** (IntBox, StrBox) — doesn't scale - **Any** — gives up type safety entirely - **Generics** — ONE class, type-safe for ANY type

The Solution

```
from typing import TypeVar, Generic

T = TypeVar("T")  # "some type" – placeholder

class Box(Generic[T]):
    def __init__(self, item: T):
        self.item = item
    def get(self) -> T:
        return self.item

int_box = Box[int](42)      # T = int
str_box = Box[str]("hello") # T = str

val = int_box.get()  # IDE knows: int
val2 = str_box.get() # IDE knows: str
val2.upper()         # IDE autocompletes string methods!
```

Generic Functions

```
T = TypeVar("T")

def first(items: list[T]) -> T:
    return items[0]

first([1, 2, 3])      # returns int
first(["a", "b"])     # returns str
# The return type FOLLOWS the input type.
```


Real-World: Generic Repository

```
class Repository(Generic[T]):
    def __init__(self) -> None:
        self._items: dict[int, T] = {}

    def add(self, id: int, item: T) -> None:
        self._items[id] = item

    def get(self, id: int) -> T | None:
        return self._items.get(id)

user_repo = Repository[User]()    # only accepts User
order_repo = Repository[Order]()  # only accepts Order

user_repo.add(1, some_order)      # mypy error! Expected User, got
                                   # Order.
```

One class. Type-safe for any model. This is how Django QuerySets, SQLAlchemy sessions, and FastAPI dependencies work under the hood.

Generic Stack

```
class Stack(Generic[T]):
    def __init__(self) -> None:
        self._items: list[T] = []

    def push(self, item: T) -> None:
        self._items.append(item)

    def pop(self) -> T:
        return self._items.pop()

    def peek(self) -> T:
        return self._items[-1]

int_stack = Stack[int]()
int_stack.push(10)
int_stack.push(20)
val = int_stack.pop()    # IDE knows: int

str_stack = Stack[str]()
str_stack.push("hello")
str_stack.push(42)       # mypy error: expected str, got int
```

Result Pattern — Multiple TypeVars

```
from dataclasses import dataclass
```

```
T = TypeVar("T")
E = TypeVar("E")
```

```

@dataclass
class Success(Generic[T]):
    value: T

@dataclass
class Failure(Generic[E]):
    error: E

def divide(a: float, b: float) -> Success[float] | Failure[str]:
    if b == 0:
        return Failure("Division by zero")
    return Success(a / b)

result = divide(10, 3)
if isinstance(result, Success):
    print(result.value)    # IDE knows: float
else:
    print(result.error)    # IDE knows: str

```

Where you'll see generics in the wild

- **Django:** `QuerySet[User]` — query results know the model type
- **FastAPI:** `Response[UserSchema]` — auto-generates API docs from the type
- **SQLAlchemy:** `Session.query(User)` returns typed results
- **Standard lib:** `list[T]`, `dict[K, V]`, `Optional[T]`, `Future[T]`

You've been using generics all along. `list[int]` IS `list` parameterized with `int`. `list` is `Generic[T]` under the hood. Same with `dict[K, V]`, `set[T]`.

Constrained and Bounded TypeVars

```

# Constrained: T can ONLY be int or float
Number = TypeVar("Number", int, float)

def double(x: Number) -> Number:
    return x * 2

double(5)        # ✓
double(3.14)     # ✓
double("hi")     # mypy error!

# Bounded: T must be Animal or subclass
A = TypeVar("A", bound=Animal)

def loudest(animals: list[A]) -> A:
    return max(animals, key=lambda a: len(a.speak()))

```

```
loudest([Dog("Rex"), Dog("Buddy")]) # returns Dog (not just
Animal!)
```

Protocol — Duck Typing with Type Safety

ABC forces inheritance. But what if you don't control the class?

```
from typing import Protocol

class Drawable(Protocol):
    def draw(self) -> str: ...

# These DON'T inherit from Drawable – they just have draw()
class Circle:
    def draw(self) -> str: return "○"

class Square:
    def draw(self) -> str: return "□"

def render(shapes: list[Drawable]) -> None:
    for s in shapes:
        print(s.draw())

render([Circle(), Square()]) # Works! No inheritance needed.
```

	ABC	Protocol
Requires inheritance?	Yes	No
Works with 3rd-party?	No	Yes
Runtime check?	isinstance()	@runtime_checkable
Analogy	Java interface	Go interface
Use when	You own the hierarchy	You need a contract for any class

mypy — The Type Checker (Deep Dive)

What Is mypy?

mypy is a **static type checker** for Python. It reads your code and type hints **WITHOUT running it** and finds type errors. Think of it as a compiler for Python types — except Python still runs fine without it.

Origin story: Created by **Jukka Lehtosalo** at the University of Cambridge as his PhD project (2012). **Guido van Rossum** (Python's creator) joined the project at Dropbox and helped make it the standard. Dropbox had 4 million lines of Python that kept breaking — mypy was their solution.

Installing and Running

```
# Install
pip install mypy

# Check one file
mypy your_file.py

# Check entire project
mypy src/

# Strict mode (recommended for new projects)
mypy src/ --strict

# Ignore one specific line (escape hatch)
result = sketchy_function() # type: ignore
```

What Does mypy Catch? — 5 Real Scenarios

1. Wrong argument type:

```
def add(a: int, b: int) -> int:
    return a + b

add(10, "20")
# mypy: Argument 2 to "add" has incompatible type "str"; expected
# "int"
```

2. Not handling None (the #1 production crash):

```
def find_user(uid: int) -> dict | None:
    return {1: {"name": "Alice"}}.get(uid)

user = find_user(999)
print(user["name"])
# mypy: Value of type "dict | None" is not indexable
# Fix: if user is not None: print(user["name"])
```

3. Wrong return type:

```
def get_age() -> int:
    return "twenty-five"
# mypy: Incompatible return value type (got "str", expected "int")
```

4. Non-existent attribute:

```
dog = Dog("Rex")
dog.speak() # mypy: "Dog" has no attribute "speak" (it has
            "bark")
```

5. Wrong container element:

```
items: list[str] = ["apple", "banana", 42]
# mypy: List item 2 has incompatible type "int"; expected "str"
```

mypy vs pyright

	mypy	pyright
Creator	Dropbox / Jukka Lehtosalo	Microsoft
Language	Python	TypeScript (faster)
IDE	Plugin required	Built into VS Code (Pylance)
Speed	Slower on large codebases	Much faster
Verdict	Use in CI/CD	VS Code already runs it via Pylance

You typically use **both**: pyright catches errors live in your IDE, mypy runs in CI/CD.

Configuration (pyproject.toml)

```
[tool.mypy]
python_version = "3.12"
strict = true
warn_return_any = true
warn_unused_ignores = true
disallow_untyped_defs = true

# Exclude tests from strict checking
[[tool.mypy.overrides]]
module = "tests.*"
disallow_untyped_defs = false

# Ignore third-party libs without type stubs
[[tool.mypy.overrides]]
module = "some_old_library.*"
ignore_missing_imports = true
```

How Production Code Enforces Types

Having type hints is useless if nobody checks them. Here's how teams make it impossible to merge type-broken code.

Git Hooks — “What even are these?”

You know `git commit`. But did you know git can run code **automatically** before or after certain actions?

The lifecycle of a git commit:

You type ``git commit``

- pre-commit hook runs (BEFORE commit is created)
- If hook PASSES → commit is created
- If hook FAILS → commit is REJECTED. Code not committed.
- post-commit hook runs (after, for notifications etc.)

Every git repo has a `.git/hooks/` folder with sample hooks.

Creating a pre-commit hook manually:

```
# 1. Create the file
cat > .git/hooks/pre-commit << 'EOF'
#!/bin/bash
echo "Running mypy..."
mypy src/
EOF

# 2. Make it executable
chmod +x .git/hooks/pre-commit

# Now every `git commit` runs mypy first!
```

If mypy finds an error, it returns a non-zero exit code. Git sees this and **refuses** to create the commit.

pre-commit Framework — The Easy Way

Writing raw shell scripts is fragile. Most teams use the pre-commit framework:

```
# .pre-commit-config.yaml (in project root)
repos:
  - repo: https://github.com/pre-commit/mirrors-mypy
    rev: v1.8.0
    hooks:
      - id: mypy
        additional_dependencies: [types-requests]

  - repo: https://github.com/astral-sh/ruff-pre-commit
    rev: v0.3.0
    hooks:
      - id: ruff          # linting
      - id: ruff-format   # formatting
```

```
pip install pre-commit
pre-commit install # once per repo clone
# Now every git commit auto-runs: mypy + ruff + formatting
```

CI/CD — The Safety Net

Pre-commit hooks run locally. But someone can skip them (`git commit --no-verify`). That's where CI/CD comes in.

```
# .github/workflows/type-check.yml
name: Type Check
on: [push, pull_request]

jobs:
  mypy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
      - run: pip install mypy
      - run: mypy src/ --strict
```

If mypy fails, the PR gets a red X. Most teams require all checks to pass before merging.

Three Layers of Defense

```
Developer writes code → IDE (pyright) shows warnings live
↓
git commit → pre-commit hook runs mypy → blocks bad commits
↓
git push + PR → CI pipeline runs mypy --strict → blocks bad merges
↓
Merge to main → only if ALL checks pass
```

Even if someone skips hooks with `--no-verify`, CI catches it on the PR. Type-broken code **never reaches main**.

Gradual Adoption — You Don't Type Everything Day 1

Phase	What to Do	Timeline
1	Type all new functions. Leave legacy alone.	Immediately
2	Add return types to existing public functions. Biggest ROI.	Week 1-2
3		Month 1

Phase	What to Do	Timeline
4	Type critical paths (money, auth, data).	Month 1-2
	Run <code>mypy src/</code> on every PR. No strict flag yet.	
5	Enable <code>--strict</code> . Use <code># type: ignore</code> for legacy.	Month 3+

Type Aliases and NewType

```
from typing import TypeAlias, NewType
```

```
# Alias: readable name for complex types
```

```
Coordinate: TypeAlias = tuple[float, float]
```

```
StudentScores: TypeAlias = dict[str, list[int]]
```

```
Middleware: TypeAlias = Callable[[Request, Response], Response]
```

```
# NewType: prevents mixing up same-typed IDs
```

```
UserId = NewType("UserId", int)
```

```
OrderId = NewType("OrderId", int)
```

```
def get_user(uid: UserId) -> User: ...
```

```
get_user(UserId(42))    # ✓
```

```
get_user(OrderId(42))  # mypy error! Can't mix up IDs.
```

```
get_user(42)           # mypy error! Must wrap with UserId()
```

Best Practices

DO Type

- All public function signatures (parameters + return)
- Class `__init__` parameters and attributes
- Variables where type isn't obvious (empty containers)
- Library/API boundaries

DON'T Type

- Obvious assignments (`name = "Alice"` — mypy infers `str`)
- Test files (diminishing returns)
- Throwaway scripts
- Every local variable inside a function

Tools

Tool	What It Does
mypy	Standard type checker. Add to CI.
pyright	Microsoft's checker. Powers VS Code/Pylance. Faster.
ruff	Fast linter, catches some type issues
pydantic	Runtime validation. Used by FastAPI.

Golden Rule

Type your **boundaries** (public APIs, function signatures, class interfaces).
Let inference handle the **internals**.

Code Files

File	What It Demonstrates
01_no_types_problem.py	The bug that takes 30 minutes without types
02_basic_type_hints.py	Basic type annotations, why they matter
03_collection_types.py	list, dict, tuple, set with element types
04_optional_union.py	Optional, Union, handling None
05_typed_dict_dataclass.py	TypedDict vs dataclass vs Pydantic
06_callable_types.py	Typing functions that accept functions
07_generics_problem.py	Why we need generics (Box problem)
08_generics_solution.py	TypeVar, Generic class, generic functions
09_generic_real_world.py	Repository pattern, Result type, Stack
10_protocol.py	Structural typing (duck typing + types)
11_mypy_demo.py	Intentional errors for mypy to catch
12_constrained_typevar.py	Constrained and bounded TypeVars
13_type_alias.py	Type aliases and NewType
14_best_practices.py	When to type, when not to, tools
15_callable_explained.py	Callable explained simply — functions as objects, typing them

Key Takeaways

1. **Types are documentation that mypy verifies** — they can't go stale like comments

2. **TypeVar = type placeholder** — T flows through your code, preserving type info. `list[int]` is already a generic!
3. **Protocol = duck typing made safe** — structural typing without forced inheritance
4. **mypy + Git hooks + CI** — three layers of defense ensure type-broken code never reaches main
5. **Gradual adoption** — type boundaries first, internals later, strict mode over time

Next class: Python Advanced-2 — Collections (`defaultdict`, `Counter`, `deque`, `namedtuple`, and when each outperforms plain list/dict).